

UNIT 4 : Lab Experiments

Slot C

Name : K. Lahari

Reg No. : 192325118

Course : Cryptography And Network Security

Course Code : CSA5104

Program 1: Caesar Cipher for Basic Secure Message Transmission

Aim

To implement the Caesar Cipher algorithm in Python for encrypting and decrypting a message, thereby demonstrating a basic technique for secure message transmission.

Algorithm

1. Start the program.
2. Read the plaintext message and the shift key (an integer).
3. For encryption, for each character in the plaintext: if it is an uppercase letter, shift it by 'key' positions within A-Z using modulo 26; if lowercase, shift within a-z; otherwise leave it unchanged.
4. Concatenate all shifted characters to form the ciphertext.
5. For decryption, repeat the same process using a shift of (-key) on the ciphertext to recover the original message.
6. Display the original message, the encrypted message and the decrypted message.
7. Stop the program.

Program (Python)

```
def caesar_encrypt(text, shift):
    result = ""
    for ch in text:
        if ch.isupper():
            result += chr((ord(ch) - 65 + shift) % 26 + 65)
        elif ch.islower():
            result += chr((ord(ch) - 97 + shift) % 26 + 97)
        else:
            result += ch
    return result

def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)

if __name__ == "__main__":
    message = "Secure Message Transmission"
    shift = 3
    print("Original Message :", message)
    print("Shift Value      :", shift)
    enc = caesar_encrypt(message, shift)
    print("Encrypted Message :", enc)
    dec = caesar_decrypt(enc, shift)
    print("Decrypted Message :", dec)
```

```
Original Message : Secure Message Transmission
Shift Value : 3
Encrypted Message : Vhfxuh Phvvdjh Wudqvplvvlrq
Decrypted Message : Secure Message Transmission
```

Program 2: Vigenère Cipher for Encrypting and Decrypting Web Communication Data

Aim

To implement the Vigenère Cipher algorithm to encrypt and decrypt data exchanged over web communication using a repeating keyword.

Algorithm

1. Start the program.
2. Read the plaintext and the keyword.
3. Generate a full-length key by repeating the keyword to match the length of the plaintext.
4. For encryption, for each pair of plaintext character (P) and key character (K), compute Cipher = (P + K) mod 26, mapped back to an uppercase letter.
5. For decryption, for each pair of cipher character (C) and key character (K), compute Plain = (C - K + 26) mod 26.
6. Display the plaintext, generated key, ciphertext and decrypted text.
7. Stop the program.

Program (Python)

```
def generate_key(text, key):
    key = list(key)
    if len(text) == len(key):
        return "".join(key)
    key = (key * (len(text) // len(key) + 1))[:len(text)]
    return "".join(key)

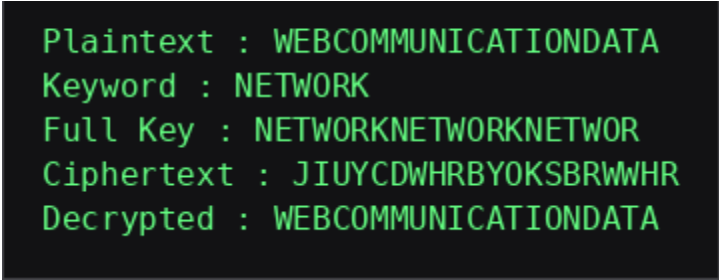
def vigenere_encrypt(text, key):
    result = []
    for t, k in zip(text, key):
        result.append(chr((ord(t) + ord(k)) % 26 + 65))
    return "".join(result)

def vigenere_decrypt(cipher, key):
    result = []
    for c, k in zip(cipher, key):
        result.append(chr((ord(c) - ord(k) + 26) % 26 + 65))
    return "".join(result)

if __name__ == "__main__":
    text = "WEBCOMMUNICATIONDATA"
    keyword = "NETWORK"
    key = generate_key(text, keyword)
    print("Plaintext :", text)
    print("Keyword   :", keyword)
    print("Full Key    :", key)
```

```
cipher = vigenere_encrypt(text, key)
print("Ciphertext :", cipher)
decrypted = vigenere_decrypt(cipher, key)
print("Decrypted :", decrypted)
```

Output Screenshot



```
Plaintext : WEBCOMMUNICATIONDATA
Keyword : NETWORK
Full Key : NETWORKNETWORKNETWOR
Ciphertext : JIUYCDWHRBYOKSBRWWHR
Decrypted : WEBCOMMUNICATIONDATA
```

Program 3: RSA Encryption and Decryption for Secure Client-Server Communication

Aim

To implement the RSA public-key cryptosystem to encrypt and decrypt a message exchanged between a client and a server.

Algorithm

1. Start the program.
2. Choose two large distinct prime numbers p and q .
3. Compute $n = p * q$ and Euler's totient $\phi(n) = (p-1)*(q-1)$.
4. Choose public exponent e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$ (commonly 65537).
5. Compute private exponent d as the modular multiplicative inverse of e modulo $\phi(n)$.
6. Public key = (e, n) ; Private key = (d, n) .
7. For encryption (client side), compute $\text{cipher_char} = (\text{plain_char})^e \bmod n$ for every character.
8. For decryption (server side), compute $\text{plain_char} = (\text{cipher_char})^d \bmod n$ for every value.
9. Display keys, plaintext, ciphertext and decrypted text.
10. Stop the program.

Program (Python)

```
import random
from sympy import isprime, mod_inverse

def generate_keys(bits=16):
    def rand_prime():
        while True:
            n = random.randrange(2**(bits-1), 2**bits)
            if isprime(n):
                return n
    p, q = rand_prime(), rand_prime()
    while p == q:
        q = rand_prime()
    n = p * q
    phi = (p - 1) * (q - 1)
```

```

e = 65537
if phi % e == 0:
    e = 3
d = mod_inverse(e, phi)
return (e, n), (d, n), p, q

def encrypt(pub, msg):
    e, n = pub
    return [pow(ord(ch), e, n) for ch in msg]

def decrypt(priv, cipher):
    d, n = priv
    return "".join(chr(pow(c, d, n)) for c in cipher)

if __name__ == "__main__":
    public_key, private_key, p, q = generate_keys()
    print("Prime p :", p)
    print("Prime q :", q)
    print("Public Key (e, n):", public_key)
    print("Private Key (d, n):", private_key)
    message = "HI RSA"
    print("Plaintext (Client -> Server):", message)
    ciphertext = encrypt(public_key, message)
    print("Ciphertext:", ciphertext)
    decrypted = decrypt(private_key, ciphertext)
    print("Decrypted (Server side):", decrypted)

```

Output Screenshot

```

Prime p : 42929
Prime q : 53129
Public Key (e, n): (65537, 2280774841)
Private Key (d, n): (1750607233, 2280774841)
Plaintext (Client -> Server): HI RSA
Ciphertext: [250684871, 1808187780, 1431589139, 1197426015, 622574961, 731793055]
Decrypted (Server side): HI RSA

```

Program 4: Diffie-Hellman Key Exchange and Shared Secret Key Generation

Aim

To implement the Diffie-Hellman Key Exchange algorithm and demonstrate how two parties (Alice and Bob) can generate an identical shared secret key over an insecure channel.

Algorithm

1. Start the program.
2. Choose and publicly agree upon a large prime number P and a primitive root/base G .
3. Alice selects a private key 'a' and computes her public key $A = G^a \text{ mod } P$.
4. Bob selects a private key 'b' and computes his public key $B = G^b \text{ mod } P$.
5. Alice and Bob exchange their public keys A and B over the (insecure) channel.
6. Alice computes the shared secret as $B^a \text{ mod } P$.
7. Bob computes the shared secret as $A^b \text{ mod } P$.
8. Verify that both computed shared secrets are equal.

9. Stop the program.

Program (Python)

```
import random

def power(base, exp, mod):
    return pow(base, exp, mod)

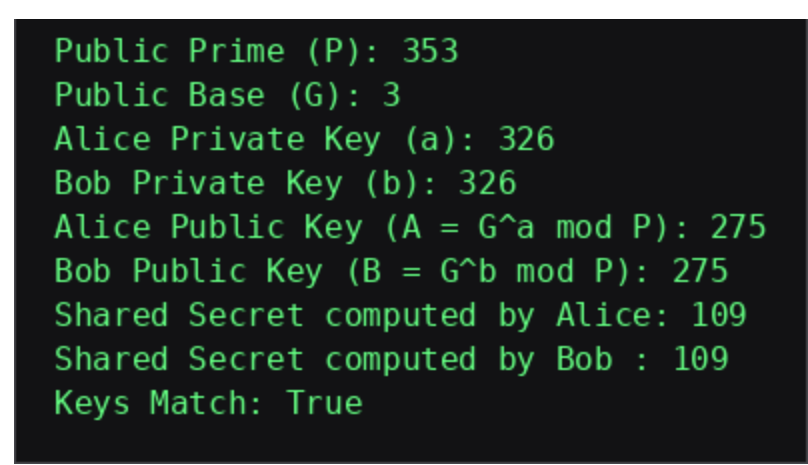
if __name__ == "__main__":
    P = 353 # public prime
    G = 3   # public base (primitive root of P)
    print("Public Prime (P):", P)
    print("Public Base (G):", G)

    a = random.randint(2, P - 2) # Alice private
    b = random.randint(2, P - 2) # Bob private
    print("Alice Private Key (a):", a)
    print("Bob Private Key (b):", b)

    A = power(G, a, P) # Alice public
    B = power(G, b, P) # Bob public
    print("Alice Public Key (A = G^a mod P):", A)
    print("Bob Public Key (B = G^b mod P):", B)

    shared_alice = power(B, a, P)
    shared_bob = power(A, b, P)
    print("Shared Secret computed by Alice:", shared_alice)
    print("Shared Secret computed by Bob :", shared_bob)
    print("Keys Match:", shared_alice == shared_bob)
```

Output Screenshot



```
Public Prime (P): 353
Public Base (G): 3
Alice Private Key (a): 326
Bob Private Key (b): 326
Alice Public Key (A = G^a mod P): 275
Bob Public Key (B = G^b mod P): 275
Shared Secret computed by Alice: 109
Shared Secret computed by Bob : 109
Keys Match: True
```

Program 5: SHA-256 Hashing to Verify Integrity of a Web Document

Aim

To implement the SHA-256 hashing algorithm to verify the integrity of a web document or message and detect any unauthorized modification.

Algorithm

1. Start the program.
2. Read the original document/message text.
3. Compute its SHA-256 hash digest using the hashlib library and store it as the reference hash.
4. When the document is received/re-checked, compute the SHA-256 hash of the received content.
5. Compare the newly computed hash with the reference hash.
6. If both hashes match, declare the document as unmodified (integrity verified); otherwise declare it as tampered.
7. Repeat the check with a deliberately tampered copy of the document to demonstrate detection.
8. Stop the program.

Input

Document = "This is the official web document content." and a tampered version of the same text

Program (Python)

```
import hashlib

def sha256_hash(data):
    return hashlib.sha256(data.encode()).hexdigest()

if __name__ == "__main__":
    original_doc = "This is the official web document content."
    original_hash = sha256_hash(original_doc)
    print("Original Document :", original_doc)
    print("SHA-256 Hash      :", original_hash)

    received_doc = "This is the official web document content."
    received_hash = sha256_hash(received_doc)
    print("\nReceived Document :", received_doc)
    print("Computed Hash      :", received_hash)
    print("Integrity Check   :", "VERIFIED - Document Unmodified" if original_hash == received_hash else "FAILED - Document Tampered")

    tampered_doc = "This is the official web document content!"
    tampered_hash = sha256_hash(tampered_doc)
    print("\nTampered Document :", tampered_doc)
    print("Computed Hash      :", tampered_hash)
    print("Integrity Check   :", "VERIFIED" if original_hash == tampered_hash else "FAILED - Document Tampered")
```

Output Screenshot

```
Original Document : This is the official web document content.
SHA-256 Hash : 9568d9ac7442dc7588ab6937a78df1e728e4a9665b2a87070a6e0681cb3b3048
Received Document : This is the official web document content.
Computed Hash : 9568d9ac7442dc7588ab6937a78df1e728e4a9665b2a87070a6e0681cb3b3048
Integrity Check : VERIFIED - Document Unmodified
Tampered Document : This is the official web document content!
Computed Hash : 5a9aeed8adc0bbb5e4762336ffbed8813db92da3e30e4e0eaf6655571b8bbd74
Integrity Check : FAILED - Document Tampered
```

Program 6: HMAC using SHA-256 for Message Authentication and Integrity

Aim

To implement HMAC (Hash-based Message Authentication Code) using SHA-256 to verify the authenticity and integrity of a transmitted message using a shared secret key.

Algorithm

1. Start the program.
2. Read the shared secret key and the message to be authenticated.
3. Compute the HMAC-SHA256 value of the message using the secret key with Python's hmac module.
4. Send the message along with the computed HMAC value to the receiver.
5. At the receiver side, recompute the HMAC over the received message using the same shared key.
6. Compare the recomputed HMAC with the received HMAC using a constant-time comparison.
7. If they match, the message is authentic and unaltered; otherwise reject the message.
8. Stop the program.

Input

Key = "SharedSecretKey123", Message = "Transfer \$500 to Account 45213"

Program (Python)

```
import hmac
import hashlib

def generate_hmac(key, message):
    return hmac.new(key.encode(), message.encode(), hashlib.sha256).hexdigest()

def verify_hmac(key, message, received_mac):
    expected = generate_hmac(key, message)
    return hmac.compare_digest(expected, received_mac)

if __name__ == "__main__":
    secret_key = "SharedSecretKey123"
    message = "Transfer $500 to Account 45213"
    mac = generate_hmac(secret_key, message)
    print("Shared Key :", secret_key)
    print("Message :", message)
    print("HMAC-SHA256:", mac)

    print("\n-- Receiver Side --")
```

```
print("Verification with correct message:", verify_hmac(secret_key, message, mac))

tampered_message = "Transfer $5000 to Account 45213"
print("Verification with tampered message:", verify_hmac(secret_key, tampered_message, mac))
```

Output Screenshot

```
Shared Key : SharedSecretKey123
Message : Transfer $500 to Account 45213
HMAC-SHA256: c56f0146d6b4b50c041917e833ebd003206c1dd7631e6cced778e225c2b961aa
-- Receiver Side --
Verification with correct message: True
Verification with tampered message: False
```

Program 7: Generate and Verify a Digital Signature using RSA

Aim

To develop a program that generates and verifies a digital signature using the RSA algorithm, ensuring message authenticity, integrity and non-repudiation.

Algorithm

1. Start the program.
2. Generate an RSA key pair - public key (e, n) and private key (d, n).
3. Compute the SHA-256 hash of the message to be signed.
4. Generate the digital signature by encrypting the hash with the sender's private key: $\text{signature} = \text{hash}^d \bmod n$.
5. Send the message and signature to the receiver, along with the sender's public key.
6. At the receiver, decrypt the received signature using the sender's public key: $\text{recovered_hash} = \text{signature}^e \bmod n$.
7. Independently compute the hash of the received message.
8. If the recovered hash equals the independently computed hash, the signature is valid; otherwise it is invalid.
9. Stop the program.

Input

Message = "Approve Fund Transfer Order #778" (16-bit random RSA primes)

Program (Python)

```
import hashlib
import random
from sympy import isprime, mod_inverse

def generate_keys(bits=16):
    def rand_prime():
        while True:
            n = random.randrange(2**(bits-1), 2**bits)
            if isprime(n):
                return n
    p, q = rand_prime(), rand_prime()
    while p == q:
        q = rand_prime()
```

```

n = p * q
phi = (p - 1) * (q - 1)
e = 65537 if phi % 65537 != 0 else 3
d = mod_inverse(e, phi)
return (e, n), (d, n)

def sign(message, private_key):
    d, n = private_key
    h = int(hashlib.sha256(message.encode()).hexdigest(), 16) % n
    return pow(h, d, n)

def verify(message, signature, public_key):
    e, n = public_key
    h = int(hashlib.sha256(message.encode()).hexdigest(), 16) % n
    h_from_sig = pow(signature, e, n)
    return h == h_from_sig

if __name__ == "__main__":
    public_key, private_key = generate_keys()
    print("Public Key (e, n):", public_key)
    print("Private Key (d, n):", private_key)

    message = "Approve Fund Transfer Order #778"
    print("\nMessage to Sign:", message)
    signature = sign(message, private_key)
    print("Digital Signature:", signature)

    print("\nVerification (original message):", verify(message, signature, public_key))
    tampered = "Approve Fund Transfer Order #779"
    print("Verification (tampered message) :", verify(tampered, signature, public_key))

```

Output Screenshot

```

Public Key (e, n): (65537, 2611572233)
Private Key (d, n): (787620833, 2611572233)
Message to Sign: Approve Fund Transfer Order #778
Digital Signature: 741098077
Verification (original message): True
Verification (tampered message) : False

```

Program 8: Simulate a Secure Client-Server Communication System using Symmetric Encryption and Hashing

Aim

To simulate a secure client-server communication system that uses symmetric (Fernet/AES) encryption for confidentiality and SHA-256 hashing for integrity verification.

Algorithm

1. Start the program.
2. Generate a shared symmetric session key (Fernet key) known to both client and server.
3. At the client: take the plaintext message, encrypt it using the symmetric key, and compute its SHA-256 digest.
4. Send the ciphertext and digest to the server.
5. At the server: decrypt the ciphertext using the shared symmetric key to recover the plaintext.
6. Recompute the SHA-256 digest of the decrypted plaintext.
7. Compare the recomputed digest with the received digest to verify integrity.
8. Display whether the integrity check passed.
9. Stop the program.

Input

Message = "Login Successful: Session Token 8823-XZ"

Program (Python)

```
import hashlib
from cryptography.fernet import Fernet

def client_send(message, key):
    f = Fernet(key)
    encrypted = f.encrypt(message.encode())
    digest = hashlib.sha256(message.encode()).hexdigest()
    print("[CLIENT] Plaintext      :", message)
    print("[CLIENT] Encrypted (AES)  :", encrypted)
    print("[CLIENT] SHA-256 Digest   :", digest)
    return encrypted, digest

def server_receive(encrypted, digest, key):
    f = Fernet(key)
    decrypted = f.decrypt(encrypted).decode()
    recomputed = hashlib.sha256(decrypted.encode()).hexdigest()
    print("[SERVER] Decrypted      :", decrypted)
    print("[SERVER] Recomputed Hash :", recomputed)
    print("[SERVER] Integrity Check :", "OK" if recomputed == digest else "FAILED")
    return decrypted

if __name__ == "__main__":
    key = Fernet.generate_key()
    print("Symmetric Session Key:", key.decode())
    message = "Login Successful: Session Token 8823-XZ"
    enc, digest = client_send(message, key)
    print()
    server_receive(enc, digest, key)
```

Output Screenshot

```
Symmetric Session Key: 7NV-UXToEg2umf-lA_FymQQA17FyTyePNcG-jB2S9zw=
[CLIENT] Plaintext : Login Successful: Session Token 8823-XZ
[CLIENT] Encrypted (AES) : b'gAAAAABqg-MSjpkAVZpBq5RyF8XjdV_pmRYDrPC3-
euMhnN21bIGoewBUUnqpPUY7Y_sqCGC8zNYaB2Yua344Rq5JbjBjt08pGRQu0xiwq_wFodQiLZsAnWZRIVSEbXtEd
E00CSdfJyes'
[CLIENT] SHA-256 Digest : c649832933249e4b6d89e3be6aeb6f87d68a882918d2d0e131d082e8206ff34f
[SERVER] Decrypted : Login Successful: Session Token 8823-XZ
[SERVER] Recomputed Hash : c649832933249e4b6d89e3be6aeb6f87d68a882918d2d0e131d082e8206ff34f
[SERVER] Integrity Check : OK
```

Program 9: Configure a TLS-Secured Client-Server Connection and Verify the TLS Handshake and Certificate

Aim

To configure a TLS-secured socket connection between a client and a server using Python's ssl module, and to verify handshake details such as protocol version, negotiated cipher suite and the server's certificate.

Algorithm

1. Start the program.
2. Generate a self-signed certificate and private key for the server.
3. Create a server-side SSLContext, load the certificate/key, bind a TCP socket, and wrap it for TLS.
4. Start the server in a separate thread, listening for incoming secure connections.
5. Create a client-side SSLContext, load the server's certificate as a trusted CA, and connect to the server.
6. After the TLS handshake completes, retrieve and display the negotiated TLS version, cipher suite and the server certificate's subject/issuer/validity.
7. Exchange a test message over the secured channel to confirm the connection works.
8. Stop the program.

Input

Self-signed certificate (server.crt/server.key) generated for CN=localhost; test message = "Hello Server, this is the client"

Program (Python)

```
import socket
import ssl
import threading
import time

HOST, PORT = "127.0.0.1", 9443

def run_server():
    ctx = ssl.SSLServerContext = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    ctx.load_cert_chain(certfile="server.crt", keyfile="server.key")
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        sock.bind((HOST, PORT))
        sock.listen(1)
    with ctx.wrap_socket(sock, server_side=True) as ssock:
        conn, addr = ssock.accept()
```

```

        data = conn.recv(1024)
        print("[SERVER] Received:", data.decode())
        conn.send(b"Hello Client, TLS handshake successful!")
        conn.close()

def run_client():
    ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
    ctx.load_verify_locations("server.crt")
    ctx.check_hostname = False
    with socket.create_connection((HOST, PORT)) as sock:
        with ctx.wrap_socket(sock, server_hostname="localhost") as ssock:
            print("[CLIENT] TLS Version negotiated :", ssock.version())
            print("[CLIENT] Cipher Suite in use  :", ssock.cipher())
            cert = ssock.getpeercert()
            print("[CLIENT] Server Certificate Subject:", cert.get("subject"))
            print("[CLIENT] Server Certificate Issuer :", cert.get("issuer"))
            print("[CLIENT] Certificate valid until  :", cert.get("notAfter"))
            ssock.send(b"Hello Server, this is the client")
            reply = ssock.recv(1024)
            print("[CLIENT] Server Reply:", reply.decode())

if __name__ == "__main__":
    t = threading.Thread(target=run_server, daemon=True)
    t.start()
    time.sleep(0.5)
    run_client()
    t.join(timeout=1)

```

Output Screenshot

```

[CLIENT] TLS Version negotiated : TLSv1.3
[CLIENT] Cipher Suite in use : ('TLS_AES_256_GCM_SHA384', 'TLSv1.3', 256)
[CLIENT] Server Certificate Subject: (((('countryName', 'IN'),), (('stateOrProvinceName',
    'Tamil Nadu'),), (('organizationName', 'College Lab'),), (('commonName', 'localhost'),))
[CLIENT] Server Certificate Issuer : (((('countryName', 'IN'),), (('stateOrProvinceName',
    'Tamil Nadu'),), (('organizationName', 'College Lab'),), (('commonName', 'localhost'),))
[CLIENT] Certificate valid until : Aug 18 04:41:02 2027 GMT
[SERVER] Received: Hello Server, this is the client
[CLIENT] Server Reply: Hello Client, TLS handshake successful!

```

Program 10: Establish an SSL/TLS Socket Connection between a Client and Server

Aim

To develop a program that establishes a secure SSL/TLS socket connection between a client and a server program and exchanges data securely.

Algorithm

1. Start the program.
2. Load the server's certificate and private key into an SSL server context.
3. Create a TCP socket, bind it to a host/port, and wrap it with the SSL context to accept secure connections.
4. Start the server in a background thread, listening for a client connection.
5. On the client side, create an SSL client context that trusts the server's certificate.
6. Establish a plain TCP connection to the server and wrap it with the client SSL context to complete the TLS handshake.
7. Send a confidential message from the client to the server over the secured socket.
8. The server receives the message and sends back an acknowledgement.
9. Stop the program.

Input

Server host/port = 127.0.0.1:9444; message = "Confidential data over TLS socket"

Program (Python)

```
import socket
import ssl
import threading
import time

HOST, PORT = "127.0.0.1", 9444

def server():
    ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    ctx.load_cert_chain(certfile="server.crt", keyfile="server.key")
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind((HOST, PORT))
        s.listen(1)
        print("[SERVER] Listening on", (HOST, PORT))
        with ctx.wrap_socket(s, server_side=True) as tls:
            conn, addr = tls.accept()
            print("[SERVER] Secure connection established with", addr)
            msg = conn.recv(1024).decode()
            print("[SERVER] Received:", msg)
            conn.sendall(b"ACK: message received securely")
            conn.close()

def client():
    ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
    ctx.load_verify_locations("server.crt")
    ctx.check_hostname = False
    with socket.create_connection((HOST, PORT)) as sock:
        with ctx.wrap_socket(sock, server_hostname="localhost") as tls:
            time.sleep(0.2)
            print("[CLIENT] Secure socket connected. Protocol:", tls.version())
            tls.sendall(b"Confidential data over TLS socket")
            reply = tls.recv(1024)
```

```

        print("[CLIENT] Server response:", reply.decode())

if __name__ == "__main__":
    t = threading.Thread(target=server, daemon=True)
    t.start()
    time.sleep(0.5)
    client()
    t.join(timeout=1)

```

Output Screenshot

```

[SERVER] Listening on ('127.0.0.1', 9444)
[SERVER] Secure connection established with ('127.0.0.1', 32902)
[CLIENT] Secure socket connected. Protocol: TLSv1.3
[SERVER] Received: Confidential data over TLS socket
[CLIENT] Server response: ACK: message received securely

```

Program 11: Analyze an SSL/TLS Handshake and Identify ClientHello, ServerHello, Certificate, Key Exchange and Finished Messages

Aim

To analyze the sequence of messages exchanged during an SSL/TLS handshake and identify the ClientHello, ServerHello, Certificate, Key Exchange and Finished stages.

Algorithm

1. Start the program.
2. Set up a TLS server (with a self-signed certificate) and a TLS client using Python's ssl and socket modules.
3. Instrument both client and server code to log a message at each conceptual handshake stage: ClientHello, ServerHello, Certificate exchange, Key Exchange, and Finished.
4. Run the server in a background thread and initiate the client connection.
5. As the handshake proceeds, record the stage name and a description of what happens at that stage (e.g., cipher negotiated, certificate validated).
6. After the connection is established, print the complete ordered summary of handshake stages that were observed.
7. Stop the program.

Input

Localhost TLS connection using a self-signed certificate

Program (Python)

```

import socket
import ssl
import threading
import time

HOST, PORT = "127.0.0.1", 9445
handshake_log = []

```

```

def log(stage, detail):
    handshake_log.append((stage, detail))
    print(f'[{stage}] {detail}')

def server():
    ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    ctx.load_cert_chain(certfile="server.crt", keyfile="server.key")
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind((HOST, PORT))
        s.listen(1)
        log("ServerHello", "Server ready, will present Certificate + negotiate cipher suite")
        with ctx.wrap_socket(s, server_side=True) as tls:
            conn, addr = tls.accept()
            log("Finished(Server)", f"Handshake complete with {addr}, secure channel ready")
            conn.recv(64)
            conn.close()

def client():
    ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
    ctx.load_verify_locations("server.crt")
    ctx.check_hostname = False
    log("ClientHello", "Client proposes TLS version, supported cipher suites, random nonce")
    with socket.create_connection((HOST, PORT)) as sock:
        with ctx.wrap_socket(sock, server_hostname="localhost") as tls:
            log("Certificate", "Server certificate received and validated against trust store")
            log("KeyExchange", f"Session key established, cipher = {tls.cipher()[0]}")
            log("Finished(Client)", f"Handshake complete. Protocol = {tls.version()}")
            tls.send(b"done")

if __name__ == "__main__":
    t = threading.Thread(target=server, daemon=True)
    t.start()
    time.sleep(0.4)
    client()
    t.join(timeout=1)
    print("\n--- Handshake Message Sequence Summary ---")
    for i, (stage, detail) in enumerate(handshake_log, 1):
        print(f'{i}. {stage}: {detail}')

```

Output Screenshot

```

[ServerHello] Server ready, will present Certificate + negotiate cipher suite
[ClientHello] Client proposes TLS version, supported cipher suites, random nonce
[Finished(Server)] Handshake complete with ('127.0.0.1', 50970), secure channel ready
[Certificate] Server certificate received and validated against trust store
[KeyExchange] Session key established, cipher = TLS_AES_256_GCM_SHA384
[Finished(Client)] Handshake complete. Protocol = TLSv1.3
--- Handshake Message Sequence Summary ---
1. ServerHello: Server ready, will present Certificate + negotiate cipher suite
2. ClientHello: Client proposes TLS version, supported cipher suites, random nonce
3. Finished(Server): Handshake complete with ('127.0.0.1', 50970), secure channel ready
4. Certificate: Server certificate received and validated against trust store
5. KeyExchange: Session key established, cipher = TLS_AES_256_GCM_SHA384
6. Finished(Client): Handshake complete. Protocol = TLSv1.3

```

Program 12: Generate a Self-Signed X.509 Certificate and Configure it for Secure Client-Server Communication

Aim

To generate a self-signed X.509 digital certificate using the Python cryptography library and configure it for use in secure client-server communication.

Algorithm

1. Start the program.
2. Generate an RSA private/public key pair (2048-bit).
3. Build the certificate Subject and Issuer name (same for a self-signed certificate) with fields such as Country, Organization and Common Name.
4. Set the certificate's serial number, validity period (Not Before / Not After) and any extensions (e.g., Subject Alternative Name).
5. Sign the certificate using the private key and the SHA-256 hashing algorithm.
6. Save the certificate and private key to PEM files (selfsigned.crt, selfsigned.key).
7. Display the certificate's subject, issuer, serial number, validity period and key size to confirm successful generation.
8. Stop the program.

Input

Subject: CN=www.labserver.local, O=College Lab Server, C=IN; validity = 365 days

Program (Python)

```
import datetime
from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa

if __name__ == "__main__":
    key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
    subject = issuer = x509.Name([
        x509.NameAttribute(NameOID.COUNTRY_NAME, "IN"),
        x509.NameAttribute(NameOID.ORGANIZATION_NAME, "College Lab Server"),
        x509.NameAttribute(NameOID.COMMON_NAME, "www.labserver.local"),
    ])
    cert = (
        x509.CertificateBuilder()
        .subject_name(subject)
        .issuer_name(issuer)
        .public_key(key.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(datetime.datetime.now(datetime.timezone.utc))
        .not_valid_after(datetime.datetime.now(datetime.timezone.utc) + datetime.timedelta(days=365))
        .add_extension(x509.SubjectAlternativeName([x509.DNSName("www.labserver.local")]), critical=False)
        .sign(key, hashes.SHA256())
    )

    with open("selfsigned.crt", "wb") as f:
        f.write(cert.public_bytes(serialization.Encoding.PEM))
    with open("selfsigned.key", "wb") as f:
        f.write(key.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.TraditionalOpenSSL,
```

```

        encryption_algorithm=serialization.NoEncryption(),
    ))

print("Self-Signed X.509 Certificate Generated")
print("Subject      :", cert.subject.rfc4514_string())
print("Issuer       :", cert.issuer.rfc4514_string())
print("Serial Number :", cert.serial_number)
print("Valid From    :", cert.not_valid_before_utc)
print("Valid Until   :", cert.not_valid_after_utc)
print("Public Key Size:", key.key_size, "bits")
print("Signature Algorithm:", cert.signature_algorithm_oid._name)
print("\nCertificate saved to selfsigned.crt")
print("Private key saved to selfsigned.key")

```

Output Screenshot

```

Self-Signed X.509 Certificate Generated
Subject : CN=www.labserver.local,O=College Lab Server,C=IN
Issuer  : CN=www.labserver.local,O=College Lab Server,C=IN
Serial Number : 294955505913183453626312075803947758488586728143
Valid From : 2026-08-18 04:44:04+00:00
Valid Until : 2027-08-18 04:44:04+00:00
Public Key Size: 2048 bits
Signature Algorithm: sha256WithRSAEncryption
Certificate saved to selfsigned.crt
Private key saved to selfsigned.key

```

Program 13: Verify an X.509 Digital Certificate Including Issuer, Validity Period and Public Key

Aim

To develop a program that loads and verifies an X.509 digital certificate, checking its issuer, validity period, public key details and self-signature.

Algorithm

1. Start the program.
2. Load the X.509 certificate (PEM format) generated in the earlier exercise.
3. Extract and display the certificate's Subject, Issuer, Serial Number, Not-Before/Not-After dates and public key size.
4. Compare the current date/time with the certificate's validity period to determine if the certificate is currently valid.
5. Extract the certificate's public key and use it to verify the certificate's own signature over its TBS (to-be-signed) bytes, confirming the certificate has not been tampered with.
6. Report the final validity status of the certificate.
7. Stop the program.

Input

Certificate file = selfsigned.crt (generated in Program 12)

Program (Python)

```
import datetime
from cryptography import x509
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding

if __name__ == "__main__":
    with open("selfsigned.crt", "rb") as f:
        cert = x509.load_pem_x509_certificate(f.read())

    print("---- Certificate Details ----")
    print("Issuer      :", cert.issuer.rfc4514_string())
    print("Subject       :", cert.subject.rfc4514_string())
    print("Serial Number  :", cert.serial_number)
    print("Not Valid Before:", cert.not_valid_before_utc)
    print("Not Valid After :", cert.not_valid_after_utc)
    print("Public Key Alg  :", cert.signature_algorithm_oid._name)
    pub_key = cert.public_key()
    print("Public Key Size :", pub_key.key_size, "bits")

    now = datetime.datetime.now(datetime.timezone.utc)
    validity_ok = cert.not_valid_before_utc <= now <= cert.not_valid_after_utc
    print("\n---- Validity Check ----")
    print("Current Time   :", now)
    print("Within Validity Period:", validity_ok)

    print("\n---- Signature Verification (self-signed) ----")
    try:
        pub_key.verify(
            cert.signature,
            cert.tbs_certificate_bytes,
            padding.PKCS1v15(),
            cert.signature_hash_algorithm,
        )
        print("Signature Valid : YES - certificate is authentic and untampered")
    except Exception as e:
        print("Signature Valid : NO -", e)

    print("\n---- Final Result ----")
    print("Certificate Status:", "VALID" if validity_ok else "EXPIRED/NOT YET VALID")
```

Output Screenshot

```
---- Certificate Details ----
Issuer      : CN=www.labserver.local,O=College Lab Server,C=IN
Subject     : CN=www.labserver.local,O=College Lab Server,C=IN
Serial Number : 294955505913183453626312075803947758488586728143
Not Valid Before: 2026-08-18 04:44:04+00:00
Not Valid After : 2027-08-18 04:44:04+00:00
Public Key Alg : sha256WithRSAEncryption
Public Key Size : 2048 bits
---- Validity Check ----
Current Time : 2026-08-18 04:44:04.369375+00:00
Within Validity Period: True
---- Signature Verification (self-signed) ----
Signature Valid : YES - certificate is authentic and untampered
---- Final Result ----
Certificate Status: VALID
```

Program 14: Simulate the Secure Electronic Transaction (SET) Protocol

Aim

To simulate the Secure Electronic Transaction (SET) protocol, showing how a Customer, Merchant, Payment Gateway and Certificate Authority interact to complete an online payment securely.

Algorithm

1. Start the program.
2. The Customer prepares the Order Information (OI) and Payment Information (PI) for a purchase.
3. The Customer computes hash digests of OI and PI (OIMD and PIMD) and combines them into a Payment Order Message Digest (POMD).
4. The Customer sends OI + POMD to the Merchant, and PI + POMD (via the Merchant, without the Merchant reading it) to the Payment Gateway.
5. The Merchant verifies the POMD using the OIMD it can compute, without ever seeing the Payment Information.
6. The Payment Gateway verifies the POMD using the PIMD it can compute, without ever seeing the Order Information.
7. The Payment Gateway contacts the Certificate Authority to validate the digital certificates of the Customer and Merchant.
8. On successful validation, the Payment Gateway authorizes the payment and the Merchant confirms the order to the Customer.
9. Stop the program.

Input

Order Info = "Order#5567: 1x Laptop, Qty 1"; Payment Info = "Card: 4551-XXXX-XXXX-9021, Amount: \$950"

Program (Python)

```
import hashlib

def h(data):
    return hashlib.sha256(data.encode()).hexdigest()[:16]

if __name__ == "__main__":
    print("=== Secure Electronic Transaction (SET) Protocol Simulation ===\n")

    order_info = "Order#5567: 1x Laptop, Qty 1"
    payment_info = "Card: 4551-XXXX-XXXX-9021, Amount: $950"

    print("[Customer] Prepares Order Information :", order_info)
    print("[Customer] Prepares Payment Information:", payment_info)

    oi_hash = h(order_info)
    pi_hash = h(payment_info)
    print("[Customer] OIMD (hash of Order Info) :", oi_hash)
    print("[Customer] PIMD (hash of Payment Info) :", pi_hash)
    pomd = h(oi_hash + pi_hash)
    print("[Customer] POMD (Dual Signature basis) :", pomd)
    print("[Customer] --> Sends Order Info + POMD to Merchant")
    print("[Customer] --> Sends Payment Info + POMD to Payment Gateway (via Merchant, unread)\n")

    print("[Merchant] Receives Order Info + POMD")
    print("[Merchant] Cannot read Payment Info (encrypted for gateway only)")
    print("[Merchant] Verifies POMD using OIMD -> Match:", h(oi_hash + pi_hash) == pomd)
    print("[Merchant] --> Forwards Payment Info + POMD to Payment Gateway\n")
```

```
print("[Payment Gateway] Receives Payment Info + POMD")
print("[Payment Gateway] Cannot read Order Info (not sent to it)")
print("[Payment Gateway] Verifies POMD using PIMD -> Match:", h(oi_hash + pi_hash) == pomd)
print("[Payment Gateway] Contacts Certificate Authority to validate Merchant & Customer certs")

print("\n[Certificate Authority] Validates certificates of Customer, Merchant, Gateway")
print("[Certificate Authority] All certificates VALID\n")

print("[Payment Gateway] Authorizes payment, sends approval to Merchant")
print("[Merchant] Confirms order to Customer")
print("\n=== Transaction Completed Successfully via SET Protocol ===")
```

Output Screenshot

```
=== Secure Electronic Transaction (SET) Protocol Simulation ===
[Customer] Prepares Order Information : Order#5567: 1x Laptop, Qty 1
[Customer] Prepares Payment Information: Card: 4551-XXXX-XXXX-9021, Amount: $950
[Customer] OIMD (hash of Order Info) : 927fb3f5a7723343
[Customer] PIMD (hash of Payment Info) : 4d111b4d1fccc59b
[Customer] POMD (Dual Signature basis) : 5e524c7230f48057
[Customer] --> Sends Order Info + POMD to Merchant
[Customer] --> Sends Payment Info + POMD to Payment Gateway (via Merchant, unread)
[Merchant] Receives Order Info + POMD
[Merchant] Cannot read Payment Info (encrypted for gateway only)
[Merchant] Verifies POMD using OIMD -> Match: True
[Merchant] --> Forwards Payment Info + POMD to Payment Gateway
[Payment Gateway] Receives Payment Info + POMD
[Payment Gateway] Cannot read Order Info (not sent to it)
[Payment Gateway] Verifies POMD using PIMD -> Match: True
[Payment Gateway] Contacts Certificate Authority to validate Merchant & Customer certs
[Certificate Authority] Validates certificates of Customer, Merchant, Gateway
[Certificate Authority] All certificates VALID
[Payment Gateway] Authorizes payment, sends approval to Merchant
[Merchant] Confirms order to Customer
=== Transaction Completed Successfully via SET Protocol ===
```

Program 15: Implement a Simplified SET Dual-Signature Mechanism

Aim

To implement a simplified dual-signature mechanism (as used in the SET protocol) that protects Order Information and Payment Information while allowing both the Merchant and the Payment Gateway to verify the link between them.

Algorithm

1. Start the program.
2. Generate an RSA key pair for the customer.
3. Compute the hash digest of the Order Information (OIMD) and the hash digest of the Payment Information (PIMD).
4. Concatenate OIMD and PIMD and hash the result to obtain the Payment Order Message Digest (POMD).
5. Sign the POMD with the customer's private key to produce the Dual Signature.
6. At the Merchant side, using the Dual Signature, the customer's public key, OIMD and PIMD, recompute the POMD from the signature and compare it against the independently computed POMD.
7. At the Payment Gateway side, perform the same verification using PIMD, OIMD and the Dual Signature.
8. Report whether the verification succeeds at both the Merchant and Gateway.
9. Stop the program.

Input

Order Info = "OI: Laptop x1, Ship to Chennai"; Payment Info = "PI: Card ending 9021, Amount \$950"

Program (Python)

```
import hashlib
import random
from sympy import isprime, mod_inverse

def rand_prime(bits=16):
    while True:
        n = random.randrange(2**(bits-1), 2**bits)
        if isprime(n):
            return n

def generate_keys():
    p, q = rand_prime(), rand_prime()
    while p == q:
        q = rand_prime()
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 65537 if phi % 65537 != 0 else 3
    d = mod_inverse(e, phi)
    return (e, n), (d, n)

def h(data):
    return int(hashlib.sha256(data.encode()).hexdigest(), 16)

if __name__ == "__main__":
    public_key, private_key = generate_keys()
    e, n = public_key
    d, _ = private_key

    order_info = "OI: Laptop x1, Ship to Chennai"
    payment_info = "PI: Card ending 9021, Amount $950"

    oimd = h(order_info) % n
```

```

pimd = h(payment_info) % n
print("OIMD (hash of Order Info) :", oimd)
print("PIMD (hash of Payment Info):", pimd)

pomd = h(str(oimd) + str(pimd)) % n
print("POMD = H(OIMD || PIMD) :", pomd)

dual_signature = pow(pomd, d, n)
print("\nCustomer's Dual Signature (signed with customer private key):", dual_signature)

print("\n--- Merchant Side (has OI + OIMD + Dual Signature + PIMD) ---")
recovered_pomd_m = pow(dual_signature, e, n)
computed_pomd_m = h(str(oimd) + str(pimd)) % n
print("Recovered POMD from signature:", recovered_pomd_m)
print("Computed POMD using OIMD+PIMD:", computed_pomd_m)
print("Merchant Verification Result :", "VALID" if recovered_pomd_m == computed_pomd_m else "INVALID")

print("\n--- Payment Gateway Side (has PI + PIMD + Dual Signature + OIMD) ---")
recovered_pomd_g = pow(dual_signature, e, n)
computed_pomd_g = h(str(oimd) + str(pimd)) % n
print("Recovered POMD from signature:", recovered_pomd_g)
print("Computed POMD using OIMD+PIMD:", computed_pomd_g)
print("Gateway Verification Result :", "VALID" if recovered_pomd_g == computed_pomd_g else "INVALID")

```

Output Screenshot

```

OIMD (hash of Order Info) : 917337936
PIMD (hash of Payment Info): 1604807926
POMD = H(OIMD || PIMD) : 1619916339
Customer's Dual Signature (signed with customer private key): 910157153
--- Merchant Side (has OI + OIMD + Dual Signature + PIMD) ---
Recovered POMD from signature: 1619916339
Computed POMD using OIMD+PIMD: 1619916339
Merchant Verification Result : VALID
--- Payment Gateway Side (has PI + PIMD + Dual Signature + OIMD) ---
Recovered POMD from signature: 1619916339
Computed POMD using OIMD+PIMD: 1619916339
Gateway Verification Result : VALID

```

Program 16: Detect a Virus-Like File Pattern using Signature-Based Scanning

Aim

To write a program that scans files for known virus signatures (byte patterns) and reports whether each file is infected or clean, demonstrating signature-based antivirus scanning.

Algorithm

1. Start the program.
2. Build a signature database containing known malicious byte patterns/strings.
3. Create a set of sample files, some clean and some containing an embedded malicious signature (including the standard EICAR test string).
4. For each file, read its content and search for any signature from the database within it.

5. If a signature is found, mark the file as INFECTED and report the matched signature.
6. If no signature is found, mark the file as CLEAN.
7. Display the scan results for all files.
8. Stop the program.

Input

Sample files: clean_file.txt, infected_file.txt (contains 'malicious_payload_v2'), eicar_test.txt (contains EICAR test string)

Program (Python)

```
import os

VIRUS_SIGNATURES = [
    "X5O!P%@AP[4\\PZX54(P^)7CC)7}$EICAR",
    "malicious_payload_v2",
    "TROJAN.GENERIC.KILL",
]

def scan_file(path):
    with open(path, "r", errors="ignore") as f:
        content = f.read()
    for sig in VIRUS_SIGNATURES:
        if sig in content:
            return True, sig
    return False, None

if __name__ == "__main__":
    test_dir = "scan_samples"
    os.makedirs(test_dir, exist_ok=True)
    files = {
        "clean_file.txt": "This is a normal document about network security.",
        "infected_file.txt": "Some header data...malicious_payload_v2...trailing bytes",
        "eicar_test.txt": "X5O!P%@AP[4\\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE",
    }
    for name, content in files.items():
        with open(os.path.join(test_dir, name), "w") as f:
            f.write(content)

    print("Signature Database:", VIRUS_SIGNATURES)
    print("\nScanning files in:", test_dir, "\n")
    for name in files:
        path = os.path.join(test_dir, name)
        infected, sig = scan_file(path)
        status = f"INFECTED (signature matched: {sig})" if infected else "CLEAN"
        print(f"{name:20s} -> {status}")
```

Output Screenshot

```
Signature Database: ['X5O!P%@AP[4\\PZX54(P^)7CC)7}$EICAR', 'malicious_payload_v2',
'TROJAN.GENERIC.KILL']
Scanning files in: scan_samples
clean_file.txt -> CLEAN
infected_file.txt -> INFECTED (signature matched: malicious_payload_v2)
eicar_test.txt -> INFECTED (signature matched: X5O!P%@AP[4\\PZX54(P^)7CC)7}$EICAR)
```

Program 17: Demonstrate Worm Propagation in a Controlled Simulated Network

Aim

To develop a program that simulates how a worm propagates through a network of interconnected nodes and identifies which nodes become infected over successive rounds.

Algorithm

1. Start the program.
2. Create a simulated network of N nodes, where each node is randomly connected to a few peer nodes (edges).
3. Select a 'patient zero' node and mark it as infected.
4. For each propagation round, for every infected node, attempt to infect each of its uninfected neighbours with a defined infection probability.
5. Add all newly infected nodes to the infected set.
6. Repeat the propagation rounds until either all nodes are infected or a maximum round limit is reached.
7. Display the infected nodes after every round and the final list of infected nodes.
8. Stop the program.

Input

Network of 10 nodes, each connected to up to 3 random peers; Patient Zero = Node 0; infection probability = 0.6

Program (Python)

```
import random

class Network:
    def __init__(self, num_nodes):
        self.nodes = list(range(num_nodes))
        self.infected = set()
        self.edges = {n: set() for n in self.nodes}
        for n in self.nodes:
            peers = random.sample([x for x in self.nodes if x != n], k=min(3, num_nodes - 1))
            for p in peers:
                self.edges[n].add(p)
                self.edges[p].add(n)

    def infect(self, node):
        self.infected.add(node)

    def propagate_round(self, infection_prob=0.6):
        newly_infected = set()
        for node in list(self.infected):
            for neighbor in self.edges[node]:
                if neighbor not in self.infected and random.random() < infection_prob:
                    newly_infected.add(neighbor)
        self.infected.update(newly_infected)
        return newly_infected

if __name__ == "__main__":
    random.seed(7)
    net = Network(10)
    print("Network Topology (node: connected peers):")
    for n in net.nodes:
        print(f" Node {n}: {sorted(net.edges[n])}")

    patient_zero = 0
    net.infect(patient_zero)
```

```

print(f"\n[Round 0] Worm introduced at Node {patient_zero}")
print("Infected Nodes:", sorted(net.infected))

round_num = 1
while len(net.infected) < len(net.nodes) and round_num <= 6:
    new = net.propagate_round()
    print(f"\n[Round {round_num}] Newly Infected Nodes: {sorted(new) if new else 'None'}")
    print(f"Total Infected So Far: {sorted(net.infected)}")
    round_num += 1

print(f"\nFinal Infected Nodes ({len(net.infected)}/{len(net.nodes)}):", sorted(net.infected))

```

Output Screenshot

```

Network Topology (node: connected peers):
Node 0: [1, 3, 4, 5, 6, 8, 9]
Node 1: [0, 2, 4, 5, 7]
Node 2: [1, 3, 9]
Node 3: [0, 2, 4, 5, 7, 9]
Node 4: [0, 1, 3, 7, 9]
Node 5: [0, 1, 3, 7]
Node 6: [0, 7, 8, 9]
Node 7: [1, 3, 4, 5, 6]
Node 8: [0, 6, 9]
Node 9: [0, 2, 3, 4, 6, 8]
[Round 0] Worm introduced at Node 0
Infected Nodes: [0]
[Round 1] Newly Infected Nodes: [3, 4, 5, 6, 9]
Total Infected So Far: [0, 3, 4, 5, 6, 9]
[Round 2] Newly Infected Nodes: [1, 2, 7, 8]
Total Infected So Far: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
Final Infected Nodes (10/10): [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

```

Program 18: Trojan Detection Simulation Based on Suspicious File/Process Characteristics

Aim

To develop a program that analyzes a list of running processes and flags potential Trojan-infected processes based on suspicious characteristics such as hidden status, unusual network ports, startup persistence and high CPU usage.

Algorithm

1. Start the program.
2. Maintain a list of process records, each with attributes: name, listening port, startup persistence flag, CPU usage, and hidden flag.
3. Maintain a set of known suspicious/backdoor port numbers.
4. For each process, compute a risk score by adding points for: listening on a suspicious port, being hidden from normal listing, being both hidden and persistent at startup, and abnormally high CPU usage.
5. Record the specific reasons contributing to the risk score.

6. If the total risk score exceeds a threshold, classify the process as 'TROJAN SUSPECTED'; otherwise classify it as 'SAFE'.
7. Display the risk score, verdict and reasons for every process.
8. Stop the program.

Input

5 sample processes including one masquerading as 'update_helper.exe' listening on port 4444

Program (Python)

```
processes = [
    {"name": "chrome.exe", "port": None, "startup": False, "cpu": 12, "hidden": False},
    {"name": "svchost.exe", "port": None, "startup": True, "cpu": 3, "hidden": False},
    {"name": "update_helper.exe", "port": 4444, "startup": True, "cpu": 45, "hidden": True},
    {"name": "explorer.exe", "port": None, "startup": True, "cpu": 8, "hidden": False},
    {"name": "systemhealth.exe", "port": 6667, "startup": True, "cpu": 2, "hidden": True},
]

SUSPICIOUS_PORTS = {4444, 6667, 1337, 31337}

def analyze(proc):
    score = 0
    reasons = []
    if proc["port"] in SUSPICIOUS_PORTS:
        score += 40
        reasons.append(f"listens on known malicious port {proc['port']}")
    if proc["hidden"]:
        score += 30
        reasons.append("process hides itself from normal listing")
    if proc["startup"] and proc["hidden"]:
        score += 20
        reasons.append("auto-starts at boot while hidden (persistence)")
    if proc["cpu"] > 40:
        score += 10
        reasons.append("abnormally high CPU usage")
    return score, reasons

if __name__ == "__main__":
    print("Process".ljust(20), "Risk Score", "Verdict")
    print("-" * 60)
    for p in processes:
        score, reasons = analyze(p)
        verdict = "TROJAN SUSPECTED" if score >= 50 else "SAFE"
        print(f"{p['name']:<20} {score:^10} {verdict}")
        if reasons:
            for r in reasons:
                print("    ", r)
```

Output Screenshot

```
Process Risk Score Verdict
-----
chrome.exe 0 SAFE
svchost.exe 0 SAFE
update_helper.exe 100 TROJAN SUSPECTED
- listens on known malicious port 4444
- process hides itself from normal listing
- auto-starts at boot while hidden (persistence)
- abnormally high CPU usage
explorer.exe 0 SAFE
systemhealth.exe 90 TROJAN SUSPECTED
- listens on known malicious port 6667
- process hides itself from normal listing
- auto-starts at boot while hidden (persistence)
```

Program 19: File Integrity Monitoring System using SHA-256 Hashes

Aim

To implement a simple File Integrity Monitoring (FIM) system that uses SHA-256 hashes to detect unauthorized modifications to files in a monitored directory.

Algorithm

1. Start the program.
2. Select a directory to monitor and create sample files inside it.
3. Build a baseline database by computing and storing the SHA-256 hash of every file in the directory.
4. Simulate an unauthorized modification by altering the content of one of the monitored files.
5. Re-scan the directory: for each file in the baseline, recompute its current SHA-256 hash (or detect if it has been deleted).
6. Compare the current hash with the baseline hash for every file.
7. If the hashes differ, raise an ALERT for that file (modified); if the file is missing, raise an ALERT (deleted); otherwise report the file as unchanged (OK).
8. Stop the program.

Input

Monitored directory containing config.txt and readme.txt; config.txt is modified after baseline is captured (a line 'backdoor_user=admin123' is appended)

Program (Python)

```
import hashlib
import os
import time

def sha256_of_file(path):
    with open(path, "rb") as f:
        return hashlib.sha256(f.read()).hexdigest()

def build_baseline(directory):
    baseline = {}
    for fname in os.listdir(directory):
        fpath = os.path.join(directory, fname)
        if os.path.isfile(fpath):
            baseline[fname] = sha256_of_file(fpath)
    return baseline

def check_integrity(directory, baseline):
    for fname, old_hash in baseline.items():
        fpath = os.path.join(directory, fname)
        if not os.path.exists(fpath):
            print(f"[ALERT] {fname}: FILE DELETED")
            continue
        new_hash = sha256_of_file(fpath)
        if new_hash != old_hash:
            print(f"[ALERT] {fname}: MODIFIED (old={old_hash[:12]}... new={new_hash[:12]}...)")
        else:
            print(f"[OK] {fname}: Unchanged ({new_hash[:12]}...)")

if __name__ == "__main__":
    directory = "monitor_dir"
    os.makedirs(directory, exist_ok=True)
    with open(os.path.join(directory, "config.txt"), "w") as f:
        f.write("server_mode=secure\nmax_connections=100\n")
```

```

with open(os.path.join(directory, "readme.txt"), "w") as f:
    f.write("System readme file - do not edit.")

print("Building baseline hash database...")
baseline = build_baseline(directory)
for fname, h in baseline.items():
    print(f" {fname}: {h}")

time.sleep(0.5)
print("\nSimulating unauthorized modification to config.txt...")
with open(os.path.join(directory, "config.txt"), "a") as f:
    f.write("backdoor_user=admin123\n")

print("\nRunning integrity check...")
check_integrity(directory, baseline)

```

Output Screenshot

```

Building baseline hash database...
readme.txt: 85d6a68aa0fa4fe765948f28774e101f3995937f863e7133424dba48a597b234
config.txt: bcf53344552461d6df20c1f77815996329f54d0f1d7f7f6ddf8361999ead71f8
Simulating unauthorized modification to config.txt...
Running integrity check...
[OK] readme.txt: Unchanged (85d6a68aa0fa...)
[ALERT] config.txt: MODIFIED (old=bcf533445524... new=11b71b4a04fb...)

```

Program 20: Configure a Packet-Filtering Firewall - Blocking/Allowing Traffic by IP and Port

Aim

To configure and simulate a packet-filtering firewall that allows or denies network packets based on a rule table matching source IP address and destination port number.

Algorithm

1. Start the program.
2. Define an ordered rule table, where each rule specifies an action (ALLOW/DENY), a source IP/subnet and (optionally) a destination port.
3. Define a default policy (DENY) that applies when no rule matches an incoming packet.
4. For each incoming test packet (source IP, destination port), check the rules in order.
5. If a rule's source IP/subnet and port match the packet, apply that rule's action and stop checking further rules.
6. If no rule matches, apply the default DENY policy.
7. Display the decision (ALLOW/DENY) for every test packet.
8. Stop the program.

Input

Rule table allowing HTTP/HTTPS from 192.168.1.0/24, denying 10.0.0.5 and Telnet(23) from anywhere; 5 test packets

Program (Python)

```
rules = [
    {"action": "ALLOW", "src": "192.168.1.0/24", "port": 80},
    {"action": "ALLOW", "src": "192.168.1.0/24", "port": 443},
    {"action": "DENY", "src": "10.0.0.5", "port": None},
    {"action": "DENY", "src": "ANY", "port": 23},
]

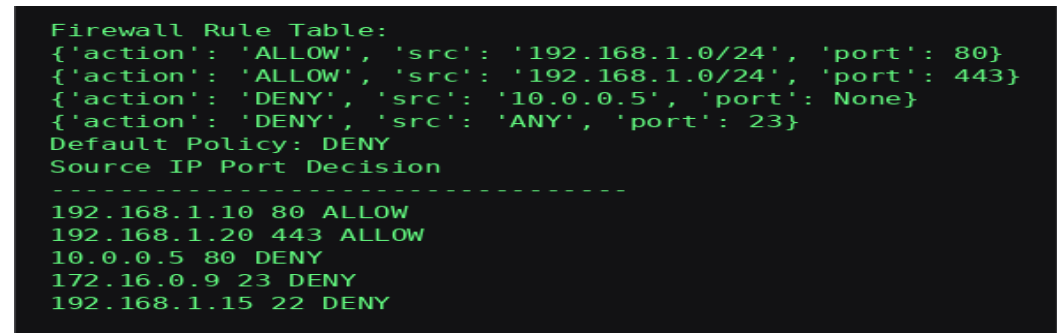
def ip_in_subnet(ip, subnet):
    if subnet == "ANY":
        return True
    if "/" not in subnet:
        return ip == subnet
    base = subnet.split("/")[0].rsplit(".", 1)[0]
    return ip.startswith(base)

def filter_packet(src_ip, dst_port):
    for rule in rules:
        if ip_in_subnet(src_ip, rule["src"]) and (rule["port"] is None or rule["port"] == dst_port):
            return rule["action"]
    return "DENY" # default deny

test_packets = [
    ("192.168.1.10", 80),
    ("192.168.1.20", 443),
    ("10.0.0.5", 80),
    ("172.16.0.9", 23),
    ("192.168.1.15", 22),
]

if __name__ == "__main__":
    print("Firewall Rule Table:")
    for r in rules:
        print(" ", r)
    print("\nDefault Policy: DENY\n")
    print(f'{"Source IP":<16} {"Port":<8} {"Decision"}')
    print("-" * 34)
    for ip, port in test_packets:
        decision = filter_packet(ip, port)
        print(f'{"ip":<16} {"port":<8} {"decision"}')
```

Output Screenshot



```
Firewall Rule Table:
{'action': 'ALLOW', 'src': '192.168.1.0/24', 'port': 80}
{'action': 'ALLOW', 'src': '192.168.1.0/24', 'port': 443}
{'action': 'DENY', 'src': '10.0.0.5', 'port': None}
{'action': 'DENY', 'src': 'ANY', 'port': 23}
Default Policy: DENY
Source IP Port Decision
-----
192.168.1.10 80 ALLOW
192.168.1.20 443 ALLOW
10.0.0.5 80 DENY
172.16.0.9 23 DENY
192.168.1.15 22 DENY
```

Program 21: Stateful Firewall Simulation Maintaining Connection State

Aim

To implement a stateful firewall simulation that tracks the state of TCP connections (using a connection/state table) and filters packets based on whether they belong to an established, tracked session.

Algorithm

1. Start the program.
2. Maintain a connection table that records the state (SYN_SENT, ESTABLISHED, etc.) of each connection, keyed by (source, destination, source port, destination port).
3. When a SYN packet is seen, create a new entry in the table with state SYN_SENT and ALLOW the packet (new outbound connection).
4. When a SYN-ACK is seen, check if a matching SYN_SENT entry exists in the reverse direction; if so, update it to ESTABLISHED and ALLOW; otherwise DENY as unsolicited.
5. When an ACK or DATA packet is seen, ALLOW only if a matching ESTABLISHED entry exists in the table; otherwise DENY.
6. When a FIN packet is seen, remove the corresponding entry from the table (connection closed) and ALLOW.
7. Display the decision for a sequence of test packets, including an unsolicited packet with no matching state.
8. Stop the program.

Input

A legitimate TCP session (SYN, SYN-ACK, ACK, DATA, FIN) plus one unsolicited DATA packet from an untracked source

Program (Python)

```
class StatefulFirewall:
    def __init__(self):
        self.connection_table = {} # (src, dst, sport, dport) -> state

    def process(self, src, dst, sport, dport, flag):
        key = (src, dst, sport, dport)
        rev_key = (dst, src, dport, sport)

        if flag == "SYN":
            self.connection_table[key] = "SYN_SENT"
            return "ALLOW (new outbound connection tracked)"

        if flag == "SYN-ACK":
            if rev_key in self.connection_table and self.connection_table[rev_key] == "SYN_SENT":
                self.connection_table[rev_key] = "ESTABLISHED"
                return "ALLOW (matches tracked connection)"
            return "DENY (unsolicited SYN-ACK, no matching state)"

        if flag == "ACK":
            if key in self.connection_table and self.connection_table[key] == "ESTABLISHED":
                return "ALLOW (established connection)"
            return "DENY (ACK with no established state)"

        if flag == "DATA":
            if key in self.connection_table and self.connection_table[key] == "ESTABLISHED":
                return "ALLOW (belongs to established session)"
            return "DENY (no matching connection state)"

        if flag == "FIN":
            if key in self.connection_table:
                del self.connection_table[key]
```

```

        return "ALLOW (connection closed, state removed)"
        return "DENY (no session to close)"

    return "DENY (unknown flag)"

if __name__ == "__main__":
    fw = StatefulFirewall()
    packets = [
        ("192.168.1.10", "8.8.8.8", 5001, 80, "SYN"),
        ("8.8.8.8", "192.168.1.10", 80, 5001, "SYN-ACK"),
        ("192.168.1.10", "8.8.8.8", 5001, 80, "ACK"),
        ("192.168.1.10", "8.8.8.8", 5001, 80, "DATA"),
        ("6.6.6.6", "192.168.1.10", 4444, 5050, "DATA"),
        ("192.168.1.10", "8.8.8.8", 5001, 80, "FIN"),
    ]
    print(f'{"Src":<15} {"Dst":<12} {"SPort":<8} {"DPort":<8} {"Flag":<10} Decision'})
    print("-" * 80)
    for src, dst, sport, dport, flag in packets:
        decision = fw.process(src, dst, sport, dport, flag)
        print(f'{"src":<15} {"dst":<12} {"sport":<8} {"dport":<8} {"flag":<10} {"decision}'))

```

Output Screenshot

```

Src Dst SPort DPort Flag Decision
-----
192.168.1.10 8.8.8.8 5001 80 SYN ALLOW (new outbound connection tracked)
8.8.8.8 192.168.1.10 80 5001 SYN-ACK ALLOW (matches tracked connection)
192.168.1.10 8.8.8.8 5001 80 ACK ALLOW (established connection)
192.168.1.10 8.8.8.8 5001 80 DATA ALLOW (belongs to established session)
6.6.6.6 192.168.1.10 4444 5050 DATA DENY (no matching connection state)
192.168.1.10 8.8.8.8 5001 80 FIN ALLOW (connection closed, state removed)

```

Program 22: Application-Level/Proxy Firewall for Controlling HTTP Traffic

Aim

To configure and demonstrate an application-level (proxy) firewall that inspects HTTP requests at the application layer and blocks requests based on HTTP method, destination domain and payload content.

Algorithm

1. Start the program.
2. Define an allow-list of trusted domains, a list of blocked HTTP methods, and a list of blocked keywords/patterns (e.g., malicious file names, SQL injection strings, script tags).
3. For each incoming HTTP request (method, URL, domain, body), first check if the method is in the blocked-methods list; if so, BLOCK.
4. Check if the destination domain is present in the allow-list; if not, BLOCK.
5. Scan the URL and request body for any blocked keyword/pattern; if found, BLOCK.
6. If none of the above checks trigger, ALLOW the request to pass through the proxy.
7. Display the decision and reason for every test request.
8. Stop the program.

Input

6 sample HTTP requests including a malware download, an SQL-injection attempt, a disallowed domain and a TRACE method request

Program (Python)

```
BLOCKED_KEYWORDS = ["malware.exe", "drop table", "<script>", "/etc/passwd"]
BLOCKED_METHODS = ["TRACE", "CONNECT"]
ALLOWED_DOMAINS = ["example.com", "college-portal.edu"]
```

```
def inspect_http_request(method, url, domain, body=""):
    if method in BLOCKED_METHODS:
        return "BLOCK", f"HTTP method '{method}' not permitted by proxy policy"
    if domain not in ALLOWED_DOMAINS:
        return "BLOCK", f"Domain '{domain}' not in allow-list"
    full = (url + " " + body).lower()
    for kw in BLOCKED_KEYWORDS:
        if kw.lower() in full:
            return "BLOCK", f"Payload contains blocked pattern: '{kw}'"
    return "ALLOW", "Request passed application-layer inspection"
```

```
requests_ = [
    ("GET", "/index.html", "example.com", ""),
    ("POST", "/login", "example.com", "user=admin&pass=1234"),
    ("GET", "/download/malware.exe", "example.com", ""),
    ("GET", "/data", "evil-site.com", ""),
    ("POST", "/search", "college-portal.edu", "q=1' OR DROP TABLE users"),
    ("TRACE", "/", "example.com", ""),
]
```

```
if __name__ == "__main__":
    print("Allowed Domains:", ALLOWED_DOMAINS)
    print("Blocked Keywords:", BLOCKED_KEYWORDS)
    print("Blocked Methods:", BLOCKED_METHODS)
    print("\n" + f"{'Method':<8} {'URL':<25} {'Domain':<18} Decision")
    print("-" * 90)
    for method, url, domain, body in requests_:
        decision, reason = inspect_http_request(method, url, domain, body)
        print(f"{'method':<8} {'url':<25} {'domain':<18} {decision} - {reason}")
```

Output Screenshot

```
Allowed Domains: ['example.com', 'college-portal.edu']
Blocked Keywords: ['malware.exe', 'drop table', '<script>', '/etc/passwd']
Blocked Methods: ['TRACE', 'CONNECT']
Method URL Domain Decision
-----
GET /index.html example.com ALLOW - Request passed application-layer inspection
POST /login example.com ALLOW - Request passed application-layer inspection
GET /download/malware.exe example.com BLOCK - Payload contains blocked pattern:
'malware.exe'
GET /data evil-site.com BLOCK - Domain 'evil-site.com' not in allow-list
POST /search college-portal.edu BLOCK - Payload contains blocked pattern: 'drop table'
TRACE / example.com BLOCK - HTTP method 'TRACE' not permitted by proxy policy
```

Program 23: Network Intrusion Detection System (NIDS) using Predefined Rules

Aim

To develop a simple Network Intrusion Detection System (NIDS) that inspects captured network packets and raises alerts when packets match predefined suspicious-activity rules.

Algorithm

1. Start the program.
2. Define a set of detection rules (e.g., port scan detection, SQL injection pattern, large ICMP packet/ping-flood, Telnet access attempt), each expressed as a condition on packet fields.
3. Prepare a stream/list of sample captured packets with fields such as source IP, destination port, protocol, flags, size and payload.
4. For each packet, evaluate it against every rule's condition.
5. If one or more rule conditions match, raise an alert naming the matched rule(s); otherwise classify the packet as normal traffic.
6. Display the detection result for every packet in the stream.
7. Stop the program.

Input

5 sample packets including a port-scan pattern, an SQL-injection payload, an oversized ICMP packet and a Telnet connection attempt

Program (Python)

```
import re

RULES = [
    {"name": "Port Scan Detection", "condition": lambda p: p["flag"] == "SYN" and p["dst_port"] in range(1, 1024) and p["repeated"] > 5},
    {"name": "SQL Injection Attempt", "condition": lambda p: bool(re.search(r"(union select|or 1=1|drop table)", p.get("payload", ""))).lower()),
    {"name": "Suspicious Large ICMP (Ping Flood)", "condition": lambda p: p["protocol"] == "ICMP" and p["size"] > 1000},
    {"name": "Telnet Access Attempt", "condition": lambda p: p["dst_port"] == 23},
]

packets = [
    {"src": "192.168.1.5", "dst_port": 80, "protocol": "TCP", "flag": "SYN", "repeated": 1, "size": 60, "payload": ""},
    {"src": "203.0.113.9", "dst_port": 22, "protocol": "TCP", "flag": "SYN", "repeated": 12, "size": 60, "payload": ""},
    {"src": "198.51.100.4", "dst_port": 80, "protocol": "TCP", "flag": "PSH", "repeated": 1, "size": 300, "payload": "id=5 UNION SELECT username,password FROM users"},
    {"src": "192.0.2.7", "dst_port": 0, "protocol": "ICMP", "flag": "-", "repeated": 1, "size": 1500, "payload": ""},
    {"src": "10.10.10.10", "dst_port": 23, "protocol": "TCP", "flag": "SYN", "repeated": 1, "size": 60, "payload": ""},
]

if __name__ == "__main__":
    print("Loaded Detection Rules:")
    for r in RULES:
        print(" -", r["name"])
    print()
    for pkt in packets:
        alerts = [r["name"] for r in RULES if r["condition"](pkt)]
        status = "; ".join(alerts) if alerts else "No match - Normal traffic"
        print(f'[Packet] src={pkt["src"]}<15} proto={pkt["protocol"]<5} dport={pkt["dst_port"]<5} -> {status}')
```

Output Screenshot

```
Loaded Detection Rules:
- Port Scan Detection
- SQL Injection Attempt
- Suspicious Large ICMP (Ping Flood)
- Telnet Access Attempt
[Packet] src=192.168.1.5 proto=TCP dport=80 -> No match - Normal traffic
[Packet] src=203.0.113.9 proto=TCP dport=22 -> Port Scan Detection
[Packet] src=198.51.100.4 proto=TCP dport=80 -> SQL Injection Attempt
[Packet] src=192.0.2.7 proto=ICMP dport=0 -> Suspicious Large ICMP (Ping Flood)
[Packet] src=10.10.10.10 proto=TCP dport=23 -> Telnet Access Attempt
```

Program 24: Intrusion Prevention System (IPS) Simulation - Detect and Block Attack Signatures

Aim

To develop an Intrusion Prevention System (IPS) simulation that not only detects packets matching known attack signatures but also actively blocks the offending source IP addresses from further communication.

Algorithm

1. Start the program.
2. Maintain a database of known attack signatures (e.g., SQL injection, XSS, command injection, port-scan patterns) and an initially empty IP blocklist.
3. For each incoming packet, first check if its source IP is already present in the blocklist; if so, DROP the packet immediately.
4. Otherwise, scan the packet's payload against every known attack signature.
5. If a signature matches, add the source IP to the blocklist and mark the packet as BLOCKED, reporting which attack type was detected.
6. If no signature matches, FORWARD the packet as legitimate traffic.
7. Display the decision for every packet and the final blocklist of IP addresses actively being dropped.
8. Stop the program.

Input

5 sample packets including SQL injection, XSS and command-injection payloads, plus a repeat packet from an already-blocked attacker

Program (Python)

```
ATTACK_SIGNATURES = {
    "SQLI": "union select",
    "XSS": "<script>",
    "CMD_INJ": "; rm -rf",
    "PORT_SCAN": "SYN_FLOOD",
}

blocked_ips = set()

def inspect_and_block(pkt):
```

```

src = pkt["src"]
if src in blocked_ips:
    return "DROPPED", "Source IP already blocked by IPS"
for attack, sig in ATTACK_SIGNATURES.items():
    if sig.lower() in pkt.get("payload", "").lower():
        blocked_ips.add(src)
        return "BLOCKED", f"Signature match: {attack} -> IP {src} added to blocklist"
return "FORWARDED", "No attack signature matched"

packets = [
    {"src": "192.168.1.5", "payload": "GET /home HTTP/1.1"},
    {"src": "203.0.113.9", "payload": "SELECT * FROM users WHERE id=1 union select pass FROM admin"},
    {"src": "198.51.100.4", "payload": "<script>alert('xss')</script>"},
    {"src": "203.0.113.9", "payload": "GET /home HTTP/1.1"}, # same attacker retries
    {"src": "10.0.0.9", "payload": "ping ; rm -rf /"},
]

if __name__ == "__main__":
    print("Attack Signature Database:", ATTACK_SIGNATURES)
    print()
    for pkt in packets:
        result, reason = inspect_and_block(pkt)
        print(f"[Packet from {pkt['src']}<15] {result}<10} - {reason}")
    print("\nFinal Blocklist (IPS actively dropping traffic from):", sorted(blocked_ips))

```

Output Screenshot

```

Attack Signature Database: {'SQLI': 'union select', 'XSS': '<script>', 'CMD_INJ': '; rm
-rf', 'PORT_SCAN': 'SYN_FLOOD'}
[Packet from 192.168.1.5 ] FORWARDED - No attack signature matched
[Packet from 203.0.113.9 ] BLOCKED - Signature match: SQLI -> IP 203.0.113.9 added to
blocklist
[Packet from 198.51.100.4 ] BLOCKED - Signature match: XSS -> IP 198.51.100.4 added to
blocklist
[Packet from 203.0.113.9 ] DROPPED - Source IP already blocked by IPS
[Packet from 10.0.0.9 ] BLOCKED - Signature match: CMD_INJ -> IP 10.0.0.9 added to blocklist
Final Blocklist (IPS actively dropping traffic from): ['10.0.0.9', '198.51.100.4',
'203.0.113.9']

```

Program 25: Network Traffic Analysis using Wireshark - Identify HTTP, HTTPS/TLS Traffic, Suspicious Packets, Open Ports and Threats

Aim

To analyze network traffic captured using Wireshark, identify HTTP and HTTPS/TLS conversations, list the distinct ports/services in use, and flag suspicious packets or possible security threats present in the capture.

Algorithm

1. Start the program (or, in the live lab, start a Wireshark capture on the active network interface).
2. Capture live traffic while browsing to an HTTP site and an HTTPS site, then stop the capture (File -> Save, or Export Packet Dissections as CSV for programmatic analysis).
3. For each captured packet, record its source/destination address, protocol, port and a short description (as shown in Wireshark's Packet List pane).

4. Filter and count packets that belong to the TLS handshake (ClientHello/ServerHello/Certificate, port 443) versus plain HTTP (port 80).
5. Identify the set of distinct destination ports used across the capture to understand which services are active.
6. Flag packets that look anomalous - e.g., unsolicited SYN packets to unusual ports (4444, 23) from an external address - as suspicious.
7. Summarize the possible security threats indicated by the suspicious packets, and note that HTTP traffic is unencrypted while HTTPS/TLS traffic is protected.
8. Stop the program.

Input

A sample captured packet log (9 packets) representing a TLS session to a web server, an HTTP request, DNS lookup, and two unsolicited probe packets from an external IP

Program (Python)

```
# Simulated analysis of packets captured in Wireshark (exported as a packet log).
# In the actual lab, this log would come from a real capture (File -> Export Packet Dissections)
# saved as CSV, e.g. capture.csv, and this script would read it directly.

packets = [
    {"no": 1, "time": "0.000", "src": "192.168.1.10", "dst": "142.250.183.14", "proto": "TCP", "port": 443, "info": "SYN"},
    {"no": 2, "time": "0.021", "src": "142.250.183.14", "dst": "192.168.1.10", "proto": "TCP", "port": 443, "info": "SYN, ACK"},
    {"no": 3, "time": "0.022", "src": "192.168.1.10", "dst": "142.250.183.14", "proto": "TLSv1.3", "port": 443, "info": "Client Hello"},
    {"no": 4, "time": "0.045", "src": "142.250.183.14", "dst": "192.168.1.10", "proto": "TLSv1.3", "port": 443, "info": "Server Hello,
Certificate, Finished"},
    {"no": 5, "time": "0.050", "src": "192.168.1.10", "dst": "93.184.216.34", "proto": "HTTP", "port": 80, "info": "GET /index.html"},
    {"no": 6, "time": "0.089", "src": "93.184.216.34", "dst": "192.168.1.10", "proto": "HTTP", "port": 80, "info": "200 OK (text/html)"},
    {"no": 7, "time": "0.500", "src": "203.0.113.55", "dst": "192.168.1.10", "proto": "TCP", "port": 4444, "info": "SYN (unsolicited,
suspicious source)"},
    {"no": 8, "time": "0.501", "src": "203.0.113.55", "dst": "192.168.1.10", "proto": "TCP", "port": 23, "info": "SYN (Telnet probe)"},
    {"no": 9, "time": "1.200", "src": "192.168.1.10", "dst": "8.8.8.8", "proto": "DNS", "port": 53, "info": "Standard query A example.com"},
]

if __name__ == "__main__":
    print(f'{"No":<4} {"Time":<8} {"Source":<18} {"Destination":<18} {"Proto":<9} {"Port":<6} Info')
    print("-" * 100)
    for p in packets:
        print(f'{p["no"]:<4} {p["time"]:<8} {p["src"]:<18} {p["dst"]:<18} {p["proto"]:<9} {p["port"]:<6} {p["info"]}')

    print("\n--- Traffic Analysis Summary ---")
    https_pkts = [p for p in packets if p["proto"] in ("TCP", "TLSv1.3") and p["port"] == 443]
    http_pkts = [p for p in packets if p["proto"] == "HTTP"]
    open_ports = sorted({p["port"] for p in packets})
    suspicious = [p for p in packets if "suspicious" in p["info"].lower() or "probe" in p["info"].lower()]

    print(f'HTTPS/TLS packets identified : {len(https_pkts)} (port 443, TLSv1.3 handshake observed)')
    print(f'HTTP (plaintext) packets : {len(http_pkts)} (port 80 - unencrypted, credentials/data exposed)')
    print(f'Distinct ports observed : {open_ports}')
    print(f'Suspicious packets flagged : {len(suspicious)}')
    for s in suspicious:
        print(f' - Packet #{s["no"]}: {s["src"]} -> port {s["port"]} : {s["info"]}')

    print("\nPossible Security Threats Identified:")
    print(" 1. Unsolicited SYN to port 4444 from 203.0.113.55 - possible backdoor/C2 scan")
    print(" 2. Unsolicited SYN to port 23 (Telnet) from 203.0.113.55 - reconnaissance/legacy service probe")
    print(" 3. Plaintext HTTP traffic on port 80 - data transmitted without encryption")
```

Output Screenshot

```
No Time Source Destination Proto Port Info
-----
-----
1 0.000 192.168.1.10 142.250.183.14 TCP 443 SYN
2 0.021 142.250.183.14 192.168.1.10 TCP 443 SYN, ACK
3 0.022 192.168.1.10 142.250.183.14 TLSv1.3 443 Client Hello
4 0.045 142.250.183.14 192.168.1.10 TLSv1.3 443 Server Hello, Certificate, Finished
5 0.050 192.168.1.10 93.184.216.34 HTTP 80 GET /index.html
6 0.089 93.184.216.34 192.168.1.10 HTTP 80 200 OK (text/html)
7 0.500 203.0.113.55 192.168.1.10 TCP 4444 SYN (unsolicited, suspicious source)
8 0.501 203.0.113.55 192.168.1.10 TCP 23 SYN (Telnet probe)
9 1.200 192.168.1.10 8.8.8.8 DNS 53 Standard query A example.com
--- Traffic Analysis Summary ---
HTTPS/TLS packets identified : 4 (port 443, TLSv1.3 handshake observed)
HTTP (plaintext) packets : 2 (port 80 - unencrypted, credentials/data exposed)
Distinct ports observed : [23, 53, 80, 443, 4444]
Suspicious packets flagged : 2
- Packet #7: 203.0.113.55 -> port 4444 : SYN (unsolicited, suspicious source)
- Packet #8: 203.0.113.55 -> port 23 : SYN (Telnet probe)
Possible Security Threats Identified:
1. Unsolicited SYN to port 4444 from 203.0.113.55 - possible backdoor/C2 scan
2. Unsolicited SYN to port 23 (Telnet) from 203.0.113.55 - reconnaissance/legacy service
   probe
3. Plaintext HTTP traffic on port 80 - data transmitted without encryption
```